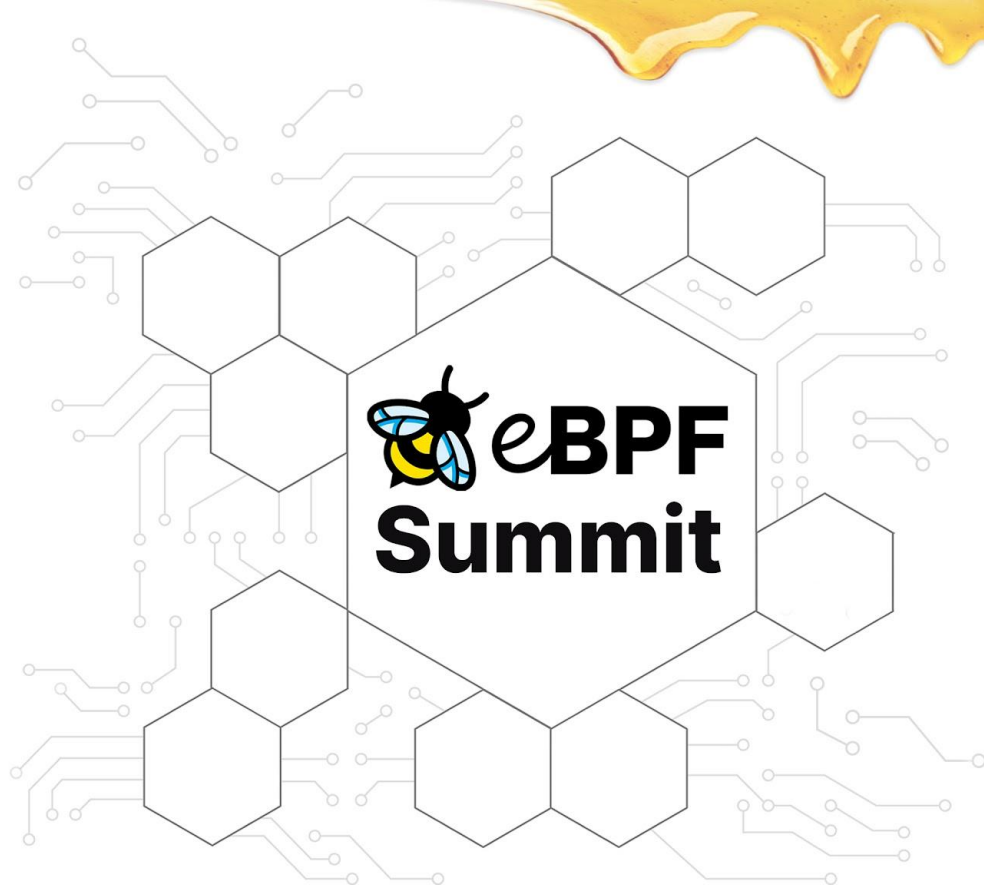


Continuous profiling with Open Telemetry

Profile everything:
All systems, all code, all the time.



Florian Lehner

 @ebpfsummit

About OTel eBPF Profiler



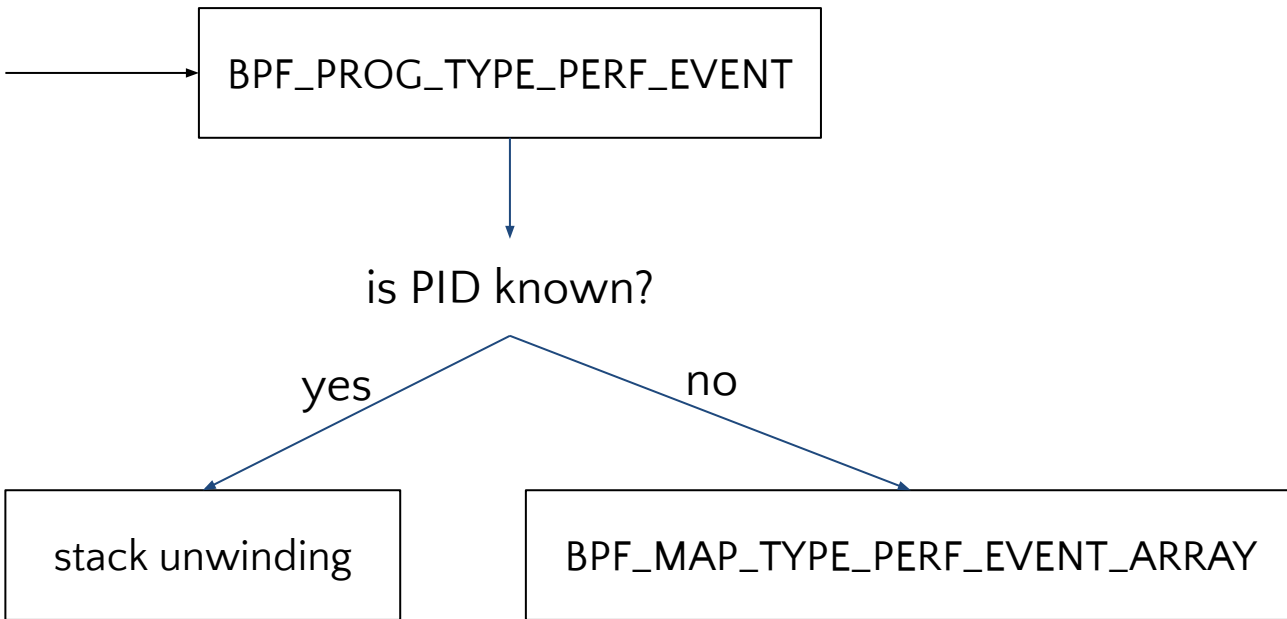
- Apache 2.0 licensed at github.com/open-telemetry/opentelemetry-ebpf-profiler
- Very low CPU and memory overhead (~ 1% CPU and ~250MB memory)
- Profiling without frame pointers and without debug symbols
- Support for native code - like C, C++, Go, Rust & others - and HLLs - like Hotspot JVM, Python, Ruby, PHP, Node.JS/V8, Perl, .NET
- Supports AMD64 and ARM64
- No reconfiguration, instrumentation or restarts required

Sampling based continuous profiling



- Leverage the [perf subsystem](#) of the Linux kernel
- Sampling rate of 20 Hz on all CPU cores
- BPF_PROG_TYPE_PERF_EVENT

Lazy loading



Memory mappings with LPM maps



- [bpf_find_vma\(\)](#) got [introduced](#) with Linux kernel 5.17
- [BPF_MAP_TYPE_LPM_TRIE](#) originates from matching IP addresses to a stored set of ranges

`/* Key of an a BPF_MAP_TYPE_LPM_TRIE entry */`

`struct bpf_lpm_trie_key {`

`__u32 prefixlen; /* up to 32 for AF_INET, 128 for AF_INET6 */`

`__u8 data[0]; /* Arbitrary size */`

`};`

Memory mappings with LPM maps



// PIDPage represents the key of the eBPF map pid_page_to_mapping_info.

typedef struct [PIDPage](#) {

u32 prefixLen; // Number of bits for pid and page that defines the
// longest prefix.

__be32 pid; // Unique ID of the process.

__be64 page; // Address to a certain part of memory within PID.

} PIDPage;



Gathering the stack



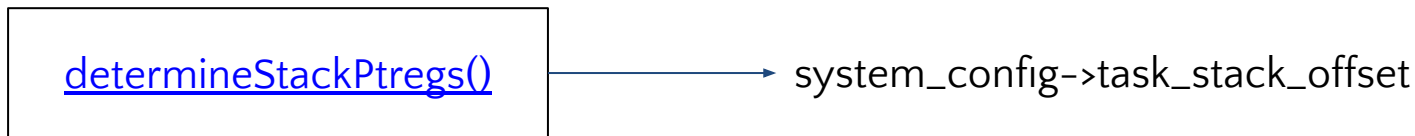
- [bpf_get_stack\(\)](#) got [introduced](#) with Linux kernel 4.18
- [bpf_task_pt_regs\(\)](#) got [introduced](#) with Linux kernel 5.15



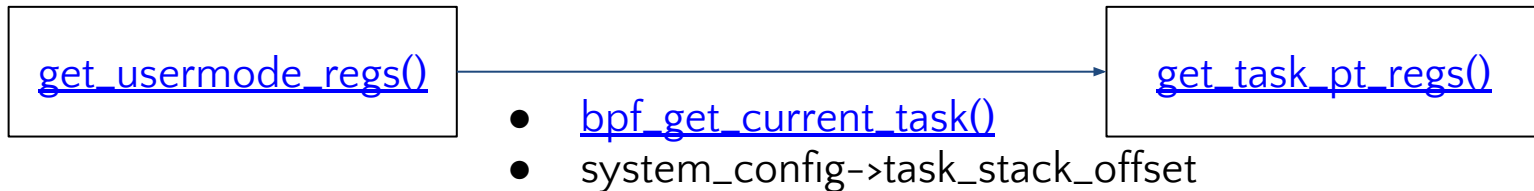
Gathering the stack



Once at startup:



For every stack unwinding:





Thank you!
Questions?